

compare_methods

December 27, 2023

Vamos a comparar los dos métodos de obtener los coeficientes de la función g_k . El primer método requiere realizar numerosas integraciones: hay que integrar la función g_k y para evaluar la función g_k hay que integrar también. El segundo método emplea el resultado 3.27 de A first Course on Waveletes

```
[ ]: MAX_K = 5
```

Método 1: integración numérica

```
[ ]: from scipy.integrate import quad
import numpy as np

def create_g_k(k):
    ck = 1/quad(lambda t: np.sin(t)**(2*k+1), 0, np.pi)[0]
    def g_k(w):
        return 1 - ck * quad(lambda t: np.sin(t)**(2*k+1), 0, w)[0]
    return g_k

def find_bk(g, k):
    '''
    Coeficientes para expresarlo como serie de exp(ikw).
    '''
    return quad(lambda t: g(t)*np.cos(k*t),
                0, 2*np.pi)[0] / (2*np.pi)

def get_coefficients_gk_1(k):
    '''
    Calcula los coeficientes de la función g_k
    cuando se escribe como serie de exp(ikw)
    '''
    gk = create_g_k(k)
    coefs = [0]*(2*(2*k+1)+1)
    for i in range(2*k+1+1):
        cc = find_bk(gk, i)
```

```

        coefs[2*k+1+i] = cc
        coefs[2*k+1-i] = cc

    return coefs

```

```
[ ]: metodo_1 = [get_coefficients_gk_1(k) for k in range(1, MAX_K)]
```

Método 2: Fórmula directa para los coeficientes de g_k

```
[ ]: from sympy import symbols, binomial, expand
import numpy as np
import re

def transform_key(key):
    '''
    Función auxiliar para transformar las claves del diccionario
    '''

    key = str(key)

    if key == '1':
        return 0

    if key == 'w':
        return 1

    match = re.search(r'w\*\*([0-9]+)', key)
    return int(match.group(1))

w = symbols('w')
def calculate_coefficients_of_gk(k):
    '''
    Devuelve coeficientes de la serie de cosenos de  $g_k$ 
    utilizando lema 3.27
    '''

    g_k = 0

    for l in range(k + 1):
        g_k += binomial(2 * k + 1, k - l) * (1 - w)**(k - l) * (1 + w)**l

    g_k *= (1 + w)**(k + 1) / (2**(2 * k + 1))

    expanded_gk = expand(g_k).as_coefficients_dict()

```

```

coeff_dict = {transform_key(k): v for k, v in
               expanded_gk.items()}

return coeff_dict

def get_conversion_dict(max_exponent):

    conversion_dict = {0: [1], 1: [1/2, 0, 1/2]}

    coef_cos = [1/2, 0, 1/2]

    for k in range(2, max_exponent+1):
        coef_cos = np.convolve(coef_cos, [1/2, 0, 1/2])
        conversion_dict[k] = coef_cos

    return conversion_dict

def transform_cos_to_exp(coeff_dict, conversion_dict=None):
    """
    Transforma los coeficientes de la serie de cosenos de  $g_k$  a una serie de
    exponenciales
    """

    if not conversion_dict:
        max_exponent = max(coeff_dict.keys())
        conversion_dict = get_conversion_dict(max_exponent)

    exponential_coefs = [0]*(2*max_exponent+1)

    for l, c in coeff_dict.items():
        j = max_exponent - l
        coefs = [0]*j + [c*v for v in conversion_dict[l]] + [0]*j
        exponential_coefs = [sum(x) for x in zip(exponential_coefs, coefs)]

    return exponential_coefs

def get_coefficients_gk_2(k):
    """
    Calcula los coeficientes de la función  $g_k$ 
    cuando se escribe como serie de  $\exp(ikw)$ 
    """

```

```
'''
```

```
cos_coefficients = calculate_coefficients_of_gk(k)
exp_coefficients = transform_cos_to_exp(cos_coefficients)
return exp_coefficients
```

```
[ ]: CONVERSION_DICT = get_conversion_dict(MAX_K)

metodo_2 = [get_coefficients_gk_2(k) for k in range(1, MAX_K)]
```

Comparación métodos:

```
[ ]: # NO TIENE EN CUENTA CUANDO ALGÚN VALOR ES CERO
```

```
def max_relative_error(list1, list2):
    max_error = 0

    for val1, val2 in zip(list1, list2):
        if val1==0 or val2==0:
            continue
        if val1 != 0:
            error = abs((val1 - val2) / val1)
            max_error = max(max_error, error)
    return max_error
```

```
def absolute_error(list1, list2):
    max_error = 0

    for val1, val2 in zip(list1, list2):
        error = abs(val1 - val2)
        max_error = max(max_error, error)
    return max_error
```

```
[ ]: for k in range(1, MAX_K):
    print(max_relative_error(metodo_1[k-1], metodo_2[k-1]),
          absolute_error(metodo_1[k-1], metodo_2[k-1]))
```

```
8.88178419700124e-16 5.55111512312578e-17
9.94759830064139e-16 7.97972798949331e-17
9.46798195400423e-14 1.66533453693773e-16
4.02979237280923e-13 1.76941794549634e-16
```

Conclusión, el primer devuelve prácticamente los coeficientes exactos.